Q3 — Data processing and cleaning (Google Sheet: Processed Data and Q3)

The supplied notebook loads the CSV into pandas, checks missing values, lists duplicate rows, creates customer age bands, converts Date to datetime, and extracts calendar fields. It then drops Customer_Age after creating Age_Group. The steps below describe the current code accurately and list additional checks that should be completed before calling the dataset analysis-ready.

Change / check	Why necessary	Risk if skipped
Missing values	**The notebook runs df.isnull().sum() to count missing values by column. The supplied code does not yet impute or remove missing values. Decide treatment column by column; do not automatically replace missing ratings or revenue with a mean. Record counts before and after treatment.**	**Unreported values can distort group summaries or cause rows to be silently excluded from specific calculations.**
Duplicate records	**The notebook creates duplicates = df[df.duplicated(keep=False)] and prints them. This lists all rows that are exact duplicates across the columns currently in df. Review them before removal. Transaction_ID may be a better key for checking duplicate transactions, but first confirm whether it is expected to be unique.**	**Duplicates may inflate transaction counts and revenue if they represent accidental repeated records.**
Date conversion	**Date is converted using pd.to_datetime(df['Date'], errors='coerce'). Invalid date strings become NaT. Count these values and decide whether to correct them from the source or exclude them from time-based analysis.**	**Invalid dates can be omitted from monthly or weekday trends and create misleading totals.**
Age categorization	**Age_Group is created with bins [0, 18, 25, 35, 45, 60, 100] and right=False. The groups are Under 18 (0–17), 18–24, 25–34, 35–44, 45–59, and 60–99. Values outside the bin range or missing ages receive a missing**	**Incorrect boundary rules or out-of-range ages can place customers in the wrong group or leave them uncategorized.**

	Age_Group and should be counted.	
Calendar fields	**Year, Month, Month_Name, Day, and Day_Name are derived from Date. These support trend analysis. Month_Name should be sorted by Month (1–12) in charts, rather than alphabetically.**	**Incorrect types or unsorted month labels can make trend charts misleading.**
Dropping Customer_Age	**The code drops Customer_Age after Age_Group is created. This can be reasonable if only age bands are needed, but retain the original age field in a raw copy so the transformation is reversible and auditable.**	**Dropping the source field too early prevents later validation or alternative grouping.**
Data types and business logic	**Check numeric types for Unit_Price, Quantity, Total_Revenue, Discount_Applied, Delivery_Time_Days, and Customer_Rating. Check whether Quantity, price, and delivery time contain impossible values. Verify how Total_Revenue is calculated before deriving a replacement.**	**Strings, impossible values, or inconsistent revenue definitions can invalidate calculations.**

Recommended reproducible checks to add to the notebook:

Record df.shape before and after processing, and save a separate raw_df copy before transformations.

Inspect df.dtypes, df.describe(include='all'), unique values for categorical fields, and missing-value percentages.

Check df['Transaction_ID'].duplicated().sum() separately from exact-row duplicates; investigate before deleting.

Check whether Total_Revenue approximately equals Unit_Price × Quantity after accounting for the meaning of Discount_Applied. Do not assume the formula until discount encoding is understood.

Check negative/zero price, quantity, revenue, delivery time, and out-of-range customer ratings. Flag unusual records for review instead of deleting automatically.

Export the final processed dataframe as a clearly named file and document every exclusion or transformation.